

Лабораторная работа №2

ООП и производительность.
Задача «Миллиард строк» (продолжение)

Содержание

1 Общие положения	2
1.1 Цель работы	2
2 Объектно-ориентированное программирование	2
2.1 Классы и объекты	2
2.2 Инкапсуляция	2
2.3 Магические методы	3
2.4 Наследование	5
2.5 Методы класса и статические методы	6
2.6 Классы данных и <code>__slots__</code>	6
3 Измерение производительности	7
3.1 Время выполнения программы	7
3.2 Профилирование	8
3.3 Построчное профилирование	8
4 Задание	9
4.1 Часть 1. Объектно-ориентированная организация решения	9
4.2 Часть 2. Производительность	9
5 Порядок сдачи	10

1 Общие положения

1.1 Цель работы

Работа продолжает лабораторную работу №1. Цель работы — изучение объектно-ориентированного программирования в Python и методов измерения производительности. По итогам работы студент должен:

- уметь проектировать классы с использованием магических методов, свойств и `dataclass`;
- уметь измерять время выполнения программы и находить узкие места с помощью профилировщика;
- понимать, от чего зависит производительность программы на Python.

2 Объектно-ориентированное программирование

2.1 Классы и объекты

```
class Point:
    dimensions = 2                # атрибут класса (аналог static)

    def __init__(self, x: float, y: float) -> None:
        self.x = x                # атрибуты экземпляра создаются присваиванием
        self.y = y

    def norm(self) -> float:
        return (self.x ** 2 + self.y ** 2) ** 0.5

p = Point(3, 4)                  # оператор new не используется
print(p.norm())                  # 5.0
print(Point.dimensions, p.dimensions)
```

- `__init__` — инициализатор. Перегрузка конструкторов не поддерживается; вместо неё используются значения аргументов по умолчанию или альтернативные конструкторы на основе `@classmethod`.
- `self` — явный аналог `this`. Он передаётся первым аргументом в каждый метод; обращение к атрибутам выполняется только через него: `self.x`, а не `x`.
- Поля в теле класса не объявляются: атрибут появляется при первом присваивании. Принято создавать все атрибуты в `__init__`.
- Деструктор в привычном для C++ смысле отсутствует; освобождение ресурсов выполняется с помощью `with`.

2.2 Инкапсуляция

Модификаторы доступа в Python отсутствуют; вместо них используются соглашения об именах:

- `self.name` — публичный атрибут (public);

- `self._name` — внутренний атрибут (protected), не предназначенный для использования вне класса. Технически он доступен, но обращение к нему извне считается нарушением соглашения;
- `self.__name` — приватный атрибут (private); использование из вне через прямое имя невозможно — имя атрибута подвергается искажению (name mangling) в `_Class__name` для предотвращения конфликтов имён в подклассах. Применяется редко.

Вместо пар методов `getX()/setX()` используются **свойства** (properties): для внешнего кода свойство выглядит как обычный атрибут, но при обращении к нему вызывается метод.

```
class Temperature:
    def __init__(self, celsius: float) -> None:
        self.celsius = celsius          # вызывается сеттер, определённый ниже

    @property
    def celsius(self) -> float:
        return self._celsius

    @celsius.setter
    def celsius(self, value: float) -> None:
        if value < -273.15:
            raise ValueError("температура ниже абсолютного нуля")
        self._celsius = value

    @property
    def fahrenheit(self) -> float:      # свойство только для чтения
        return self._celsius * 9 / 5 + 32

t = Temperature(20)
t.celsius = 25                        # проверка выполняется в сеттере
print(t.fahrenheit)                   # 77.0
# t.fahrenheit = 0                    # AttributeError: сеттер не определён
```

2.3 Магические методы

Методы, имена которых начинаются и заканчиваются двойным подчёркиванием (*dunder-методы*), вызываются интерпретатором при участии объекта во встроенных операциях. Механизм аналогичен перегрузке операторов в C++, но охватывает более широкий круг операций: преобразование в строку, получение длины, итерацию, проверку вхождения и т. д.

Метод	Вызывается при	Аналог в C++
<code>__repr__(self)</code>	<code>repr(x)</code> , в REPL, в отладчике	—
<code>__str__(self)</code>	<code>str(x)</code> , <code>print(x)</code> , <code>f"{x}"</code>	<code>operator<<</code>
<code>__eq__(self, other)</code>	<code>x == y</code>	<code>operator==</code>
<code>__lt__(self, other)</code>	<code>x < y</code> , <code>sorted()</code>	<code>operator<</code>
<code>__hash__(self)</code>	ключ <code>dict</code> , элемент <code>set</code>	<code>std::hash</code>
<code>__len__(self)</code>	<code>len(x)</code>	<code>size()</code>
<code>__getitem__(self, i)</code>	<code>x[i]</code>	<code>operator[]</code>
<code>__setitem__(self, i, v)</code>	<code>x[i] = v</code>	<code>operator[] (lvalue)</code>
<code>__contains__(self, v)</code>	<code>v in x</code>	—
<code>__iter__(self)</code>	<code>for v in x</code>	<code>begin()/end()</code>
<code>__add__(self, other)</code>	<code>x + y</code>	<code>operator+</code>
<code>__iadd__(self, other)</code>	<code>x += y</code>	<code>operator+=</code>
<code>__mul__(self, other)</code>	<code>x * y</code>	<code>operator*</code>
<code>__bool__(self)</code>	<code>if x:</code>	<code>explicit operator bool</code>

Пример — вектор на плоскости:

```
class Vec2:
    def __init__(self, x: float = 0.0, y: float = 0.0) -> None:
        self.x, self.y = x, y

    def __repr__(self) -> str:
        return f"Vec2({self.x!r}, {self.y!r})"

    def __eq__(self, other: object) -> bool:
        if not isinstance(other, Vec2):
            return NotImplemented # сравнение с этим типом не поддерживается
        return self.x == other.x and self.y == other.y

    def __add__(self, other: "Vec2") -> "Vec2":
        return Vec2(self.x + other.x, self.y + other.y)

    def __mul__(self, k: float) -> "Vec2": # v * 3
        return Vec2(self.x * k, self.y * k)

    def __rmul__(self, k: float) -> "Vec2": # 3 * v
        return self * k

    def __abs__(self) -> float:
        return (self.x ** 2 + self.y ** 2) ** 0.5

    def __iter__(self): # позволяет распаковку x, y = v
        yield self.x
        yield self.y

v = Vec2(1, 2) + 3 * Vec2(1, 0)
print(v) # Vec2(4, 2)
x, y = v
```

- `__repr__` возвращает однозначное представление объекта для разработчика, в идеале — выражение, по которому объект можно воссоздать. Метод рекомендуется определять во всех классах.
- Если `__str__` не определён, функция `print` использует `__repr__`.

- При определении `__eq__` хеш-функция по умолчанию отключается, и объект не может быть ключом словаря, пока не определён `__hash__`.
- Ключевое слово `yield` превращает функцию в *генератор*, выдающий значения по одному по мере запроса. Это наиболее простой способ реализовать `__iter__`.

2.4 Наследование

```
class Shape:
    def __init__(self, name: str) -> None:
        self.name = name

    def area(self) -> float:
        raise NotImplementedError      # аналог чисто виртуального метода

    def __repr__(self) -> str:
        return f"{type(self).__name__}({self.name!r}, area={self.area():.2f})"

class Circle(Shape):
    def __init__(self, r: float) -> None:
        super().__init__("круг")      # вызов инициализатора базового класса
        self.r = r

    def area(self) -> float:
        return 3.14159 * self.r ** 2  # переопределение метода

class Square(Shape):
    def __init__(self, a: float) -> None:
        super().__init__("квадрат")
        self.a = a

    def area(self) -> float:
        return self.a ** 2

shapes = [Circle(1), Square(2)]
print(sum(s.area() for s in shapes))
print(isinstance(shapes[0], Shape))  # True
```

Все методы в Python ведут себя как виртуальные: вызов `s.area()` всегда разрешается по фактическому классу объекта, ключевые слова `virtual` и `override` отсутствуют. Поддерживается множественное наследование; порядок поиска методов (MRO) доступен через атрибут `Circle.__mro__`.

Утиная типизация. Для использования объекта в функции не требуется его наследование от определённого класса — достаточно наличия необходимых методов. Объект с методами `__len__` и `__getitem__` поддерживает `len()` и индексацию, объект с методом `__iter__` — обход циклом `for`.

2.5 Методы класса и статические методы

```
class Date:
    def __init__(self, year: int, month: int, day: int) -> None:
        self.year, self.month, self.day = year, month, day

    @classmethod
    def from_string(cls, s: str) -> "Date":      # альтернативный конструктор
        year, month, day = (int(part) for part in s.split("-"))
        return cls(year, month, day)

    @staticmethod
    def is_leap(year: int) -> bool:              # функция в пространстве имён класса
        return year % 4 == 0 and (year % 100 != 0 or year % 400 == 0)

d = Date.from_string("2026-09-01")
Date.is_leap(2028)
```

2.6 Классы данных и __slots__

Для классов, предназначенных преимущественно для хранения данных, декоратор `@dataclass` генерирует методы `__init__`, `__repr__` и `__eq__` по аннотациям полей:

```
from dataclasses import dataclass, field

@dataclass
class Student:
    name: str
    group: int
    grades: list[int] = field(default_factory=list)    # не grades=[]

    def average(self) -> float:
        return sum(self.grades) / len(self.grades) if self.grades else 0.0

s = Student("Анна", 2)
print(s)      # Student(name='Анна', group=2, grades=[])

@dataclass(frozen=True)      # неизменяемый класс, получает __hash__
class Coord:
    lat: float
    lon: float
```

По умолчанию атрибуты объекта хранятся в словаре `obj.__dict__`, что позволяет добавлять их динамически, но увеличивает расход памяти и время доступа. Объявление `__slots__` заменяет словарь фиксированным набором полей, аналогично `struct` в C++. В примере ниже объём памяти измеряется модулем `tracemalloc` по 100 000 объектов (в значение входит и 8-байтная ссылка на объект в списке); функция `sys.getsizeof` для такого сравнения не подходит, поскольку не учитывает память, занятую атрибутами.

```
import tracemalloc
from dataclasses import dataclass

class Pixel:      # атрибуты хранятся в __dict__
```

```

def __init__(self, x: int, y: int, color: int) -> None:
    self.x, self.y, self.color = x, y, color

class SlotPixel:
    # фиксированный набор полей
    __slots__ = ("x", "y", "color")

    def __init__(self, x: int, y: int, color: int) -> None:
        self.x, self.y, self.color = x, y, color

p = SlotPixel(1, 2, 0xFF0000)
# p.alpha = 1      # AttributeError: атрибут отсутствует в __slots__

@dataclass(slots=True)
class Pixel2:
    x: int
    y: int
    color: int

def bytes_per_object(cls: type, n: int = 100_000) -> float:
    tracemalloc.start()
    # учёт всей памяти, выделяемой интерпретатором
    objects = [cls(1, 2, 3) for _ in range(n)]
    used, _ = tracemalloc.get_traced_memory()
    tracemalloc.stop()
    return used / len(objects)

print(f"Pixel:      {bytes_per_object(Pixel):.0f} байт")
print(f"SlotPixel: {bytes_per_object(SlotPixel):.0f} байт")
# Вывод (Python 3.13, 64-битная платформа):
# Pixel:      104 байт
# SlotPixel: 64 байт

```

3 Измерение производительности

3.1 Время выполнения программы

```

$ time uv run main.py measurements_10m.txt > /dev/null
real    0m4.512s      # время выполнения программы

```

Измерение выполняется несколько раз: первый запуск может быть медленнее, пока файл не загружен в кеш операционной системы. Время выполнения отдельного участка кода измеряется функцией `time.perf_counter()`: она возвращает показания монотонного таймера высокого разрешения в секундах, поэтому значение имеет только разность двух показаний.

```

import time

start = time.perf_counter()
total = sum(i * i for i in range(10_000_000))
elapsed = time.perf_counter() - start
print(f"Сумма вычислена за {elapsed:.3f} c")
# Сумма вычислена за 0.494 c

```

3.2 Профилирование

Для определения участков программы, на которые приходится основное время выполнения, используется профилировщик из стандартной библиотеки:

```
$ uv run python -m cProfile -s tottime main.py measurements_1m.txt | head -20
```

Столбец `tottime` содержит время выполнения внутри самой функции, `ncalls` — число вызовов.

Столбец `cumtime` содержит время выполнения функции вместе со всеми вызванными из неё функциями. Профилировщик сам замедляет программу, поэтому его результаты используются для поиска узких мест, а не для измерения итогового времени.

3.3 Построчное профилирование

Профилировщик `cProfile` указывает функцию, на которую приходится основное время, но не строку внутри неё. Время выполнения каждой строки функции измеряет пакет `line_profiler`. Пакет не входит в стандартную библиотеку и добавляется как зависимость для разработки:

```
$ uv add --dev line_profiler
```

Исследуемая функция помечается декоратором `@profile`:

```
from line_profiler import profile

@profile
def primes(n: int) -> list[int]:
    result = []
    for k in range(2, n):
        is_prime = True
        for d in range(2, int(k**0.5) + 1):
            if k % d == 0:
                is_prime = False
                break
        if is_prime:
            result.append(k)
    return result

if __name__ == "__main__":
    print(len(primes(100_000)))
```

Без переменной окружения `LINE_PROFILE` декоратор ничего не делает, и программа выполняется как обычно. При запуске с `LINE_PROFILE=1` результаты измерений записываются в файл `profile_output.txt` в текущем каталоге:

```
$ LINE_PROFILE=1 uv run primes.py
9592
...
Wrote profile results to profile_output.txt
```

Содержимое `profile_output.txt`:

Timer unit: 1e-09 s

Total time: 0.918304 s

File: /home/student/lab1/primes.py

Function: primes at line 4

Line #	Hits	Time	Per Hit	% Time	Line Contents
4					@profile
5					def primes(n: int) -> list[int]:
6	1	0.0	0.0	0.0	result = []
7	99999	17678000.0	176.8	1.9	for k in range(2, n):
8	99998	14904000.0	149.0	1.6	is_prime = True
9	2755285	419914000.0	152.4	45.7	for d in range(2, int(k**0.5) + 1):
10	2745693	421922000.0	153.7	45.9	if k % d == 0:
11	90406	13286000.0	147.0	1.4	is_prime = False
12	90406	13504000.0	149.4	1.5	break
13	99998	15337000.0	153.4	1.7	if is_prime:
14	9592	1755000.0	183.0	0.2	result.append(k)
15	1	4000.0	4000.0	0.0	return result

Столбец Hits содержит число выполнений строки, Time — суммарное время её выполнения в единицах Timer unit, Per Hit — среднее время одного выполнения, % Time — долю от общего времени функции. В приведённом примере более 90 % времени приходится на внутренний цикл (строки 9 и 10). Как и cProfile, построчный профилировщик замедляет программу; декораторы @profile удаляются из кода после завершения анализа.

4 Задание

Работа продолжает лабораторную работу №1 и выполняется в том же репозитории. Условие задачи «Миллиард строк», форматы входных и выходных данных и материалы каталога data/ приведены в описании лабораторной работы №1. Работа состоит из двух частей, обе обязательны.

4.1 Часть 1. Объектно-ориентированная организация решения

1. Класс StationStats (файл stats.py) хранит статистику одной станции и предоставляет:
 - метод update(value) — учёт очередного измерения;
 - метод merge(other) — объединение со статистикой другого объекта StationStats по той же станции;
 - свойства min, max, mean, count, доступные только для чтения;
 - метод __repr__, возвращающий строку min;mean;max в формате вывода.
2. Решение лабораторной работы №1 использует класс StationStats для хранения и объединения статистики станций. Интерфейс командной строки и формат вывода не меняются; check.py по-прежнему выводит OK.

4.2 Часть 2. Производительность

1. Для исходной версии решения выполняется профилирование; в README.md приводятся первые строки отчёта и вывод о том, на что приходится основное время выполнения.
2. Решение последовательно ускоряется с контролем времени выполнения. В README.md приводится таблица не менее чем для трёх версий программы: для каждой версии ука-

зываются внесённые изменения и время обработки файлов из 10 млн и 100 млн строк. Каждая версия фиксируется отдельным коммитом.

3. Все версии остаются корректными (`check.py` выводит OK) и соблюдают требования части 1.

5 Порядок сдачи

До защиты преподавателю направляется ссылка на репозиторий GitHub. Доступность ссылки проверяется без входа в учётную запись (например, в приватном окне браузера); для приватного репозитория проверяется наличие преподавателя в списке Collaborators. Сданное состояние репозитория помечается тегом `lab2` и публикуется на GitHub аналогично лабораторной работе №1; проверке подлежит именно оно.

На защите студент:

1. демонстрирует историю коммитов, содержащую отдельные версии решения;
2. запускает `check.py` на файле `measurements_100k`;
3. представляет таблицу замеров и обосновывает влияние каждого изменения на время выполнения;
4. отвечает на вопросы по коду.

Возможные вопросы

1. Является ли метод `__init__` конструктором? Для чего предназначен метод `__new__`? Каким образом в Python реализуются альтернативные конструкторы?
2. Каким образом в Python реализуется инкапсуляция? Для чего используется `@property`?
3. Что происходит, если в классе определён `__eq__`, но не определён `__hash__`?
4. Чем различаются `__repr__` и `__str__`? Какой из методов вызывает `print`?
5. Что означает `return NotImplemented` в `__eq__` или `__add__`? Для чего нужен `__rmul__`?
6. Чем `@classmethod` отличается от `@staticmethod` и от обычного метода?
7. Что такое утиная типизация? Приведите пример из стандартной библиотеки.
8. Что даёт объявление `__slots__` и какие ограничения оно накладывает?
9. Какое изменение в решении задачи дало наибольший прирост производительности и почему?

Источники

- Модель данных Python (магические методы): <https://docs.python.org/3.13/reference/datamodel.html>
- Профилировщики Python: <https://docs.python.org/3.13/library/profile.html>
- Документация `line_profiler`: <https://kernprof.readthedocs.io/en/latest/manual/index.html>
- One Billion Row Challenge: <https://github.com/gunnarmorling/1brc>